

Moemen Mahmoud Hamous

LFX Mentorship Cover Letter

WasmEdge

CNCF Cloud Native Computing Foundation

moamenhamous@gmail.com | [GitHub](#) | [LinkedIn](#)

CNCF Team.

How did you find out about our mentorship program?

- 1) I discovered your mentorship program while looking for opportunities to improve myself through learning and contributing within **open source** organizations and **real world** engineering projects. I was particularly interested in environments that encourage continuous learning, collaboration and practical experience in large scale software systems.

Why are you interested in this program?

- 2) I am highly interested in this program because it provides the opportunity to **learn** and **grow through working** within a professional **open source** environment on real world engineering problems. What particularly attracts me is that the program consistently offers project ideas that are innovative, technically challenging, and focused on solving practical problems with **real impact**, rather than repetitive or commonly implemented ideas. This strongly aligns with my interests in systems programming, problem solving, and continuously developing my technical abilities through meaningful and impactful work.

What experience and knowledge/skills do you have that are applicable to this program?

- 3) I am a **Computer Engineering** student with a strong foundation in Data Structures, Algorithms, Databases, Object Oriented Programming, and Software Design Patterns. I have hands-on experience in systems and backend development using C++ and C#. My experience includes building a **thread safe Buffer Pool Manager** in C++, where I worked on memory management, caching, concurrency control and database internals, strengthening my understanding of low level system design and performance-critical engineering. I also developed a **Bank Transaction Control System** using OOP principles, improving my system design and software architecture skills. Additionally, I built a **Smart Map Location Explorer** using ASP.NET MVC and SQL Server, gaining experience in backend development, database design, and API integration.

Experience / Open Source Contribution

LibreOffice Open Source Contributor(*Gerrit / CI Contributions*)

- 1) Contributed to LibreOffice CI pipeline (Gerrit review system)
 - 2) Worked on improving automated build/test workflows
 - 3) Participated in code review process and patch validation
 - 4) Submitted patches and collaborated with maintainers through Gerrit
 - 5) Gained experience with large-scale C++ codebases and CI systems
- [Build/Contribution:](#)

What do you hope to get out of this mentorship experience?

- 4) Through this mentorship, I aim to gain hands on experience with large scale open source systems, improve my understanding of runtime internals and system design, and strengthen my engineering discipline through collaboration with experienced mentors. My main goal is to learn from real world engineering environments, participate in discussions, and exchange knowledge with other contributors. I believe this experience will help me grow both technically and professionally, and become a more capable systems engineer able to contribute to production grade open-source projects.

Check List

- I have read the [WasmEdge Mentorship Program Policy](https://github.com/WasmEdge/WasmEdge/blob/master/docs/mentorship-policy.md) (<https://github.com/WasmEdge/WasmEdge/blob/master/docs/mentorship-policy.md>)

Pre-Test Reproduction Report:

1. Build WasmEdge from the latest master branch from source.

The WasmEdge runtime was successfully built from the latest master branch (commit: 626b35ba).

Build process completed using CMake and Make with no compilation errors.

All core components were built including:

- Executor , Loader , Validator , AOT Compiler, WASI modules, VM runtime

exact commands:

```
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge$ mkdir build
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge$ cd build
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build$ cmake ..
```

Cmake build:

```
-- Configuring done (236.0s)
-- Generating done (7.1s)
-- Build files have been written to: /mnt/c/Users/Mohamed Hamous/WasmEdge/build
```

exact commands:

```
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build$ make -j$(nproc)
```

Make build:

```
[100%] Linking CXX executable wasmedgec
[100%] Built target wasmedgec
[100%] Built target wasmedgec
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build$ git branch
* master
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build$ git log -1 --oneline
626b35ba (HEAD -> master, origin/master, origin/HEAD) test(driver): add comprehensive unit tests for subcommands
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build$
```

2.Reproduce the issue (if still reproducible) with your build in both interpreter mode and AOT mode.

1) Reproduce issue Interpreter Mode:

```
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build/tools/wasmedge$ time ./wasmedge /mnt/c/Users/Mohamed Hamous/Downloads/mutated_file_28840.wasm

real    0m0.202s
user    0m0.018s
sys     0m0.036s
```

The fuzz testcase (mutated_file_28840.wasm) was executed using Wasmedge interpreter mode built from the latest master branch (commit: 626b35ba).

Command:

```
./wasmedge mutated_file_28840.wasm
```

Observed Behavior: execution completed quickly without hanging, crashing, or producing a visible runtime error.

No trap or explicit failure message was reported.

Analysis: Although the original issue ([#2733](#)) reported a non terminating execution behavior this behavior was not reproduced in the current environment.

however, comparison with other runtimes suggests that the absence of a visible failure should not necessarily be interpreted as proof of correct execution. runtime divergence under malformed or fuzz generated inputs may indicate hidden inconsistencies in validation or memory handling paths that do not always surface as immediate crashes or hangs.

Important notes : while the testcase did not reproduce the original infinite execution symptom in interpreter mode, the execution behavior still warrants investigation due to differing outcomes across runtimes.

2) Reproduce issue – AOT Mode:

```
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build/tools/wasmedge$ ./wasmedge
compile /mnt/c/Users/Mohamed Hamous/Downloads/mutated_file_28840.wasm mutated.aot.wasm
[2026-05-25 02:57:19.280] [info] compile start
[2026-05-25 02:57:19.315] [info] verify start
[2026-05-25 02:57:19.330] [info] optimize start
[2026-05-25 02:57:20.278] [info] optimize done
[2026-05-25 02:57:20.278] [info] codegen start
[2026-05-25 02:57:20.988] [info] output start
[2026-05-25 02:57:21.090] [info] codegen done
[2026-05-25 02:57:21.095] [info] output start
[2026-05-25 02:57:21.115] [info] output done
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build/tools/wasmedge$ time ./was
medge mutated.aot.wasm

real    0m0.198s
user    0m0.014s
sys     0m0.047s
```

the testcase was successfully compiled and executed in AOT mode without hanging or producing a visible runtime trap.

this differs from the original issue report where AOT execution stalled indefinitely.

while the historical non termination behavior was not reproduced in this environment, the silent execution result does not necessarily prove complete resolution of the underlying malformed pointer condition.

given the discrepancy between Wasmedge and Wasmtime behavior further investigation may be needed to verify whether the runtime now handles the testcase differently or suppresses the fault internally.

3.1 Wasmtime Runtime Output (comparison runtime or reference runtime):

```
mohamed_hamous@DESKTOP-JVK01UB:/mnt/c/Users/Mohamed Hamous/WasmEdge/build/tools/wasmedge$ wasmtime /mnt/c/Users/Mohamed Hamous/Downloads/mutated_file_28840.wasm
Error: failed to run main module `'/mnt/c/Users/Mohamed Hamous/Downloads/mutated_file_28840.wasm`
Caused by:
  0: failed to invoke command default
  1: error while executing at wasm backtrace:
  0: 0x119ee - <unknown>!
```

Wasmtime Comparison:

Wasmtime produced an explicit runtime failure:

Pointer not aligned to 4

This indicates that the runtime detected an invalid aligned-memory access and terminated execution with a diagnostic error.

3.2 Cross-Runtime Comparison (Interpreter vs AOT vs Wasmtime)

To validate the behavior of the testcase across different WebAssembly runtimes, the same input module (mutated_file_28840.wasm) was executed under:

- Wasmedge Interpreter mode
- Wasmedge AOT mode
- Wasmtime runtime (reference implementation)

1) Wasmtime Behavior (Reference Runtime):

Wasmtime produced a **deterministic runtime trap** when executing the testcase:

Error: Pointer not aligned to 4
Execution aborted due to invalid memory alignment

This indicates that Wasmtime correctly enforces WebAssembly memory alignment rules and immediately terminates execution upon detecting the invalid memory access.

2) WasmEdge Behavior (Interpreter Mode):

In Interpreter mode, WasmEdge execution completed without emitting a clear alignment violation or runtime trap.

Observed behavior:

(no explicit alignment trap observed)
execution completed immediately

No diagnostic error indicating misaligned memory access was reported.

3) WasmEdge Behavior (AOT Mode):

In Ahead-of-Time (AOT) mode, the precompiled module executed successfully without producing a visible alignment error or runtime trap.

Observed behavior:

(no explicit alignment trap observed)
execution completed immediately

4. Expected WasmEdge Output (Interpreter & AOT)

Expected Runtime Behavior (According to WebAssembly Specification):

According to the WebAssembly specification, memory access instructions such as:

- `i32.load`
- `i32.load8_u`
- `atomic.load`

require that:

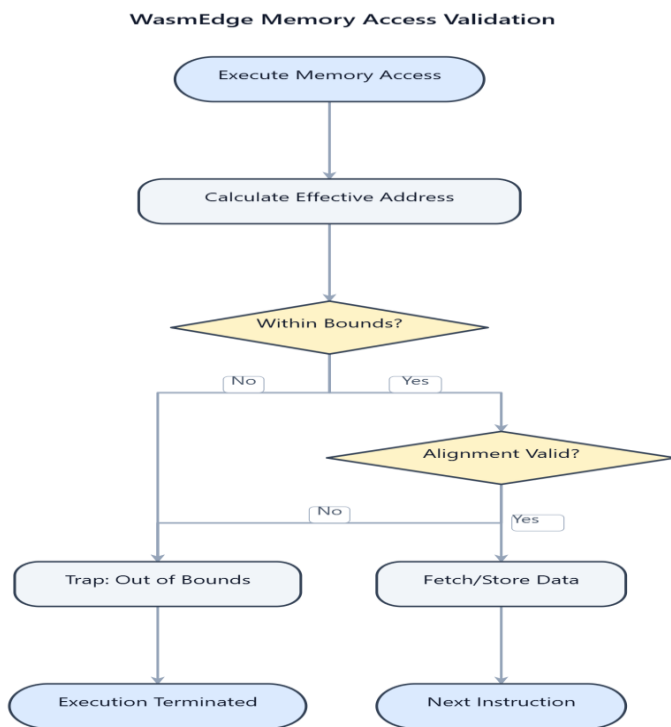
- the effective memory address must be valid within the linear memory
- and must satisfy alignment constraints defined by the instruction

When a misaligned access occurs (**address 1001 % 4 = 1 for a 4-byte load**) a conforming runtime is expected to:

Trap execution deterministically with a runtime error

This is defined behavior under WebAssembly memory safety rules and should not result in **silent continuation** or **undefined execution**.

Graphical and flow chart:



Expected WasmEdge Behavior:

1) Interpreter Mode (Expected):

If Wasmedge is strictly conforming to WebAssembly semantics, the expected behavior is:

- Detect misaligned memory access during execution
- Immediately trap execution
- Emit a clear diagnostic error

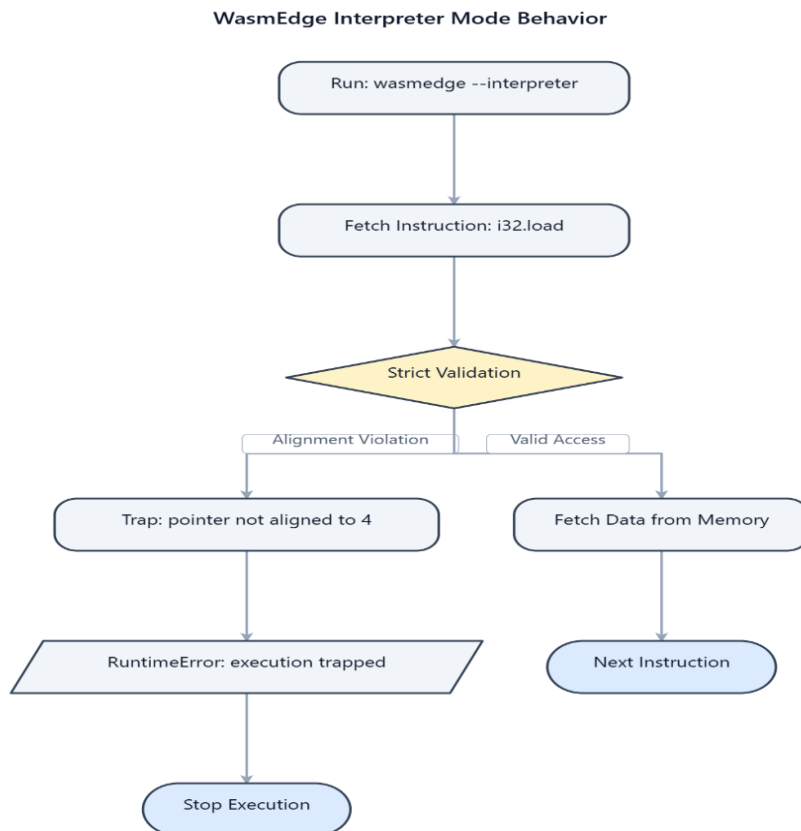
1.1 Expected output (Interpreter mode):

RuntimeError: pointer not aligned to 4
Execution trapped at instruction: i32.load

or any equivalent Wasmedge runtime diagnostic indicating:

- alignment violation
- invalid memory access trap

Graphical and flow chart:



2) Expected Behavior (AOT Mode):

In Ahead-of-Time (AOT) execution mode, Wasmedge compiles the WebAssembly module into native machine code before execution. However, it must still preserve WebAssembly semantics, especially memory safety and alignment rules.

For a misaligned memory access:

```
base = 1001
alignment requirement = 4 bytes
effective address = 1001
1001 % 4 = 1 → misaligned access
```

This violates WebAssembly alignment constraints and must be treated as an invalid memory operation.

2.1 Expected AOT Behavior:

- Either the AOT compiler detects the invalid alignment during validation and rejects execution
 - Or the generated native code includes runtime checks (guards) that detect the violation
 - In both cases, execution must terminate deterministically upon detection of the misaligned access
-

Why this behavior is expected?

This expectation is based on WebAssembly design principles:

1. Memory safety model

WebAssembly enforces strict linear memory safety. Out-of-bounds or invalid alignment access must not silently proceed.

2. Deterministic failure requirement

The specification requires failures to be:

- deterministic
- observable
- consistent across execution modes

3. Cross runtime consistency

Other runtimes such as Wasmtime correctly implement this behavior by:

- detecting misalignment
- and emitting explicit trap errors

This establishes the expected baseline behavior for conforming runtimes.

Key Conclusion:

For a valid fuzz-generated testcase involving misaligned memory access:

- Wasmedge should not silently continue execution
 - Wasmedge should not suppress alignment violations
 - Instead, it should emit a deterministic trap in both Interpreter and AOT modes
-

Proposal Timeline and Execution Plan (June 1 – August 31):

This project builds on existing issues (#2694, #2733, #2881) and focuses on reproducing and analyzing misaligned pointer behaviors, identifying root causes in execution paths, and implementing upstream fixes.

The goal is to ensure correct and deterministic trapping of malformed guest pointers in Wasmedge, and to align behavior with WASI/WITX expectations across both Interpreter and AOT modes while maintaining runtime consistency.

Weeks 1–3 — Reproduction, Runtime Audit & Root-Cause Analysis

Execution

- Reproduce issues:
 - [#2694](#) (misaligned pointer trap missing)
 - [#2733](#) (fuzz hang / non-termination)
 - [#2881](#) (misalignment reported as OOB)
- Compare runtime behavior across:
 - Wasmedge Interpreter mode
 - Wasmedge AOT mode
 - Wasmtime (reference runtime)

- Trace execution using:
 - GDB debugging
 - runtime logs
 - pointer flow analysis
- Audit WASI host pointer handling in:
 - lib/host/wasi/wasifunc.cpp

Deliverables:

- Reproduction report for all issues
 - Cross-runtime behavior comparison
 - Root-cause analysis summary
-

Weeks 4–7 — WASI Pointer Validation & Runtime Fixes:

Execution:

Implement consistent validation for guest pointers passed into WASI host functions before memory access.

Target APIs:

- `args_get`
- `args_sizes_get`
- `environ_get`
- `fd_read`
- `fd_write`
- other pointer-based WASI interfaces

Validation rule:

`effective_ptr % alignment == 0`

Ensure validation is enforced before any memory dereference.

Fix focus areas:

- Misaligned pointer handling ([#2694](#))
- Fuzz hang / non-termination ([#2733](#))
- OOB vs misalignment misclassification ([#2881](#))

Deliverables

- WASI pointer validation framework
 - Host-function validation fixes
 - Initial regression tests
-

Weeks 8–10 — Failure Propagation & Cross-Mode Consistency

Execution:

Ensure malformed pointers always produce deterministic and observable failures.

Investigate:

- silent exits
- infinite loops / hangs
- inconsistent runtime errors
- fuzz-driven divergence

Validate consistency between:

- Interpreter mode
- AOT mode

Ensure correct failure order:

alignment check → bounds check → memory access

Deliverables:

- Failure propagation improvements
 - Interpreter/AOT consistency fixes
 - Fuzz regression validation
-

Weeks 11–12 — Testing, CI & Upstream Integration

Execution:

Run full validation suite:

- WasiTest
- WebAssembly spec tests
- AOT execution tests
- fuzz regression suite

Add regression coverage for:

- aligned access (valid cases)
- misaligned access (trap cases)
- OOB cases
- combined OOB + misalignment cases

Finalize documentation and prepare upstream-ready patches.

Deliverables:

- Final regression suite
- Documentation
- Final PR set
- LFX final report

Exam Period Note:

A reduced activity window will be observed during mid-June (June 3 – June 16) due to academic exams. During this period, only lightweight documentation and reading tasks will be performed.